GROUP 3 TECHNICAL REPORT

Memory-Hierarchy-Aware Fused Attention on CUDA GPUs

Platform NVIDIA GeForce RTX 3070 Ti Laptop GPU (sm_86)
Scope Forward self-attention, BF16 storage and FP32 accumulation
Code Group 3 CUDA attention repository

1 Project overview

Driving question. When does eliminating the attention-score matrix improve actual GPU performance, and how do query tiling, parallel reduction, shared-memory layout, buffering, and Tensor Core instructions affect the result?

V0 writes an FP32 score matrix and normalises it in place. V1 fuses the three stages into one kernel and streams keys and values separately for each query row. V2a assigns eight queries to a block and shares each key and value tile across those queries. V2b retains that mapping and changes how values are laid out and read from shared memory. V3 uses BF16 Tensor Core matrix operations with a larger query tile.

V1 saves the score buffer but loses to V0 across all 60 workloads. The tiled implementations are faster on the same grid. With tiling held fixed, packing value reads cuts latency by 22–33%, compared with 1–3% for value swizzling alone. At $D = 128$, a single key buffer frees enough shared memory for a second resident block and is faster in both tiled implementations. In the full grid, V3 has median paired speedups of 57.1× over V0 and 7.1× over V2b; these compare complete implementations rather than isolating MMA alone.

2 Scope and computation

The tensors Q, K, V, O use contiguous $[B, H, N, D]$ layout with $D \in \{64, 128\}$. Within each batch and head, we compute

$$S = QK^{\mathsf{T}}/\sqrt{D}, \qquad O = \mathrm{softmax}_{\mathrm{row}}(S + M)V.$$

For dense attention $M = 0$. For causal attention $M_{ij} = -\infty$ when $j > i$ and zero otherwise. The dense dot products and value accumulation together require approximately $4BHN^2D$ floating-point operations.

2.1 Materialised scores and online softmax

V0 runs a score kernel, a stable row-wise softmax and a value-product kernel. Softmax overwrites the scores with probabilities in the same $4BHN^2$-byte workspace. V1, V2 and V3 use the online softmax approach of FlashAttention [1]. For a query, let m, ℓ, u be the running maximum, exponential sum and weighted value vector. For a tile T with valid scores s_j, the update is

$$m' = \max\left(m, \max_{j \in T} s_j\right), \qquad a = e^{m - m'},$$
$$\ell' = a\ell + \sum_{j \in T} e^{s_j - m'}, \qquad u' = au + \sum_{j \in T} e^{s_j - m'} v_j.$$

Starting from $(-\infty, 0, 0)$, each tile rescales the previous partial sums to the new maximum. The kernel writes u/ℓ after the last tile. It keeps only a tile of scores on chip and casts the final output to BF16.

3 CUDA implementation

3.1 From one query per block to one query per warp

`V0` assigns one thread to each score in a two-dimensional grid and uses shared-memory tree reductions for the row maximum and sum. In `V1`, one block owns a query, with threads computing scores against different keys. Warp shuffles perform the first step of each reduction. Warp results pass through shared memory, and block barriers separate production and consumption. The warps split the value accumulation over keys and combine their FP32 partial outputs at the end. The key tile shrinks at $D = 128$ to keep this path within its resource budget.

`V2a` and `V2b` use eight warps, each owning a different query row. The block stages 64 keys and 64 values for all eight warps. Each lane computes two scores for its query. A warp reduces the maximum and sum with shuffles, without combining results across warps. During the value product the lanes split the output dimensions, keeping two or four FP32 accumulators per lane. This mapping removes the cross-warp softmax reduction while retaining block barriers for the shared tiles. It follows the query ownership principle described in FlashAttention-2 [2].

In causal mode a V2 block stops after the tile containing its last valid query. It masks future keys per query within the final tile. For sequence tails, the copies zero-fill out-of-range keys and values, the last score tile masks out-of-range keys, and only valid query rows write outputs.

3.2 Shared-memory layout and staging

For the score product, neighbouring lanes read different rows of the K tile. A row-major BF16 layout has a stride of 128 or 256 bytes, so reading the same column group across rows repeats the bank mapping. V2 XOR-swizzles the 16-byte chunks within each key row. The row-dependent permutation distributes simultaneous reads across banks while preserving aligned copies [4]. The value product has a different access pattern. Lanes read different output dimensions from the same value row. V2a reads these values from a row-major tile. V2b uses a swizzled value tile and loads each lane's BF16 values as a packed group.

The tiled kernel uses Ampere's asynchronous global-to-shared copy [3]. It alternates two K buffers, prefetching the next keys while the current score and value work proceeds. One V buffer is filled before its value product, with the copy scheduled alongside the score work. Copy-group waits and block barriers protect each buffer before reading or reusing it.

3.3 V3: Tensor Core implementation

V3 uses BF16 `mma.m16n8k16` instructions for both products, with FP32 accumulation. Four warps handle 64 query rows per block; the key/value tile contains 64 rows at $D = 64$ and 32 at $D = 128$. Both configurations allocate 32 KiB of dynamic shared memory per block. The recorded build uses 165 and 218 registers per thread, respectively, reports no local-memory allocation, and permits three and two resident blocks per SM. The shared layout supports `ldmatrix` and asynchronous copies; online softmax operates on the score fragments. Before the second matrix multiply, unnormalised probability fragments are rounded to BF16. Thus the intermediate arithmetic differs from V2 even though matrix accumulators and softmax state are FP32.

The V3 comparison changes tile size, query mapping, reductions and staging as well as the arithmetic instructions. It measures their combined effect.

The wider head doubles the bytes in the key and value tiles. Two 54-KiB blocks exceed the SM's 100-KiB shared-memory capacity, so $D = 128$ runs with one block per SM. V2b uses the same tile storage. The single-buffer variants reduce it by one key tile.

Kernel	D	Threads/block	Keys/tile	Shared KiB/block	Max. blocks/SM
V1	64	128	64	33.0	2
V1	128	128	32	33.3	2
V2a	64	256	64	28.0	3
V2a	128	256	64	54.0	1

Table 1: Reported kernel resources. All four configurations report zero bytes of local memory per thread. Blocks per SM are residency limits.

4 Validation and experimental method

4.1 Numerical correctness

The CPU reference uses the same BF16 inputs and FP64 arithmetic for the dot products, stable softmax and value product. For each output row i, let $r_i = \max_d |O_{id}^{\text{ref}}|$. The acceptance statistic is the maximum over all rows and dimensions of

$$\frac{|O_{id} - \text{round}_{\text{BF16}}(O_{id}^{\text{ref}})|}{\text{ULP}_{\text{BF16}}(r_i)}.$$

We scale the tolerance by row so that cancellation near zero does not dominate the test. A case passes when all outputs are finite and its maximum error is at most 2 ULP.

Check	Recorded coverage
FP64, all five kernels	$N = 256$, $(B, H) = (1, 2)$, both head dimensions and masks.
FP64, ablation suite	V0, V2a, V2b and the five ablation variants at $N = 1, 7, 8, 9, 31, 32, 33, 63, 64, 65, 127, 128, 129, 256$, with $(B, H) = (1, 2)$, both head dimensions and masks.
Input and boundary cases	Gaussian, all zeros, all ones, a fixed signed value row and scaled logits. Full-output comparison includes the first and last causal rows, partial key tiles and partial query blocks.
Grid finiteness	All five kernels at $N = 128, 512, 1024, 2048, 4096$, $(B, H) \in \{(1, 8), (2, 8), (4, 8)\}$, both head dimensions and masks.

Table 2: Coverage of the archived checks. Both head dimensions are 64 and 128.

	$D = 64$		$D = 128$		
Kernel	Dense	Causal	Dense	Causal	Result
V0	0.50	0.50	0.25	0.50	Pass
V1	0.50	0.50	0.50	0.25	Pass
V2a	1.00	0.50	0.50	0.50	Pass
V2b	1.00	0.50	0.50	0.50	Pass
V3	1.00	1.00	1.00	1.00	Pass

Table 3: Maximum row-scale ULP in the archived FP64 comparisons. V1 and V3 have one main-harness case per column. V0, V2a and V2b also include 70 ablation cases per column. No failures were recorded.

The `main harness` runs the representative FP64 comparison and checks output finiteness across the performance grid. The `ablation harness` runs the tile-boundary and input-pattern checks before timing the variants; it does not include V1 or V3. For V3, the four archived main-harness cases have maximum absolute error 4.89×10^{-3} and maximum row-scaled error 5.50×10^{-3}. Grid finiteness is not an FP64 comparison.

4.2 Timing and shape coverage

We time one device-side attention call with CUDA events: three launches for V0 and one for each fused kernel, including V3. Allocation, transfers, validation and application-level latency sit outside this interval. The grid uses five warmups and 30 individual samples. The ablation harness uses ten warmups and rotates the variant order across 30 samples. We compare medians within each run and show the first and third quartiles as the sample spread.

5 Device-side attention latency

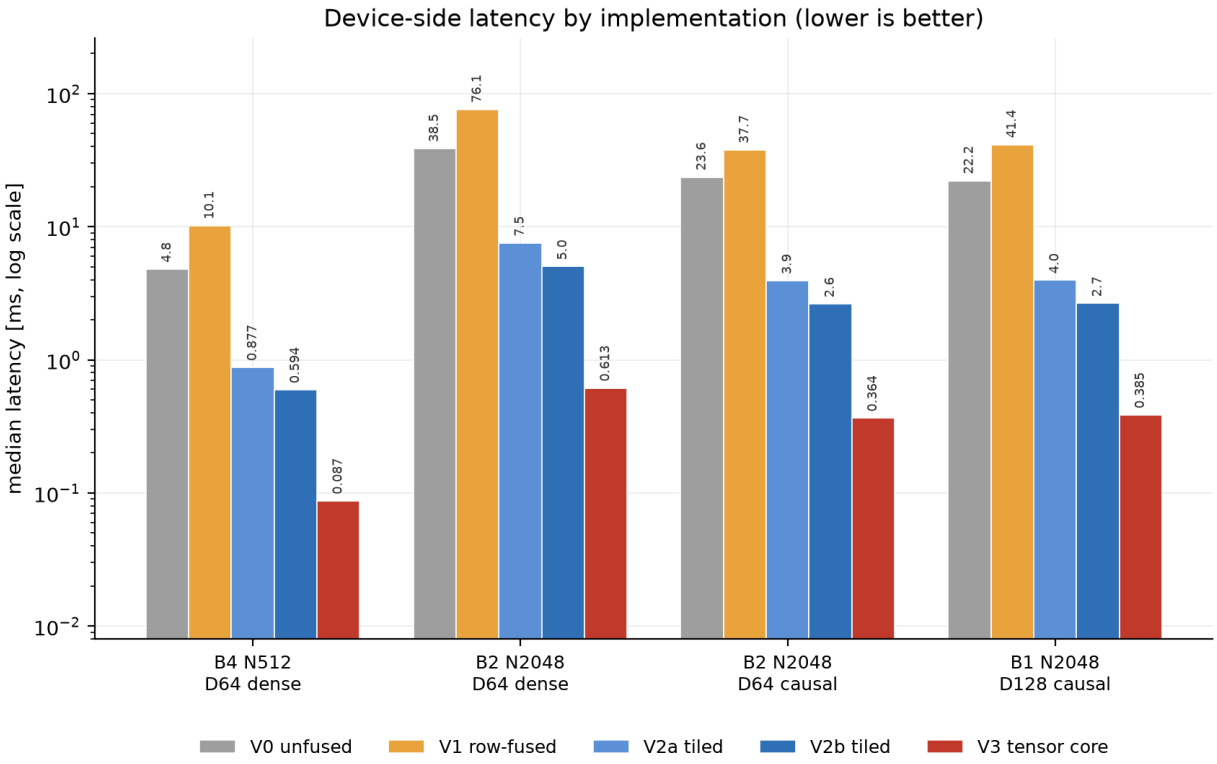


Figure 1: Device-side median latency on four representative workloads from the main grid; $H = 8$. The vertical axis is logarithmic. Raw samples and quartiles are retained in the CSV. V2b is the double-key-buffer implementation.

The full grid contains $N \in \{128, 512, 1024, 2048, 4096\}$, $D \in \{64, 128\}$, $H = 8$, $B \in \{1, 2, 4\}$ and both masks: 60 workloads. For each workload we divide its V0 median by the matching kernel median; Table 4 summarises these 60 paired ratios. V1 is slower than V0 in all 60 cases. V2a and V2b improve the query mapping and staging, while V3 is faster than V2b across the measured grid.

Comparison	Minimum	Median	Maximum
V2a / V0	4.08×	5.27×	6.84×
V2b / V0	5.89×	8.08×	10.25×
V3 / V0	9.64×	57.14×	86.36×
V3 / V2b	1.64×	7.07×	9.46×

Table 4: Speedup defined as baseline median latency divided by candidate median latency. The median column is the median of the 60 paired ratios.

At dense $(1, 8, 128, 64)$, V2b takes 18.43 μs and V3 11.26 μs, a 1.64× speedup. V3 launches only 16 blocks for this configuration, fewer than the GPU's 46 SMs. Limited grid parallelism is consistent with

its smaller short-sequence gain, although shape and work per block also change. The complete grid is preserved in the source CSV; the appendix shows the relationship between grid size and V3 throughput.

5.1 Causal tile count

For V2, let $T = N/64$ be the number of key tiles in a dense row. For the measured lengths, the average causal tile count is $(T+1)/2$, giving a causal-to-dense visit ratio of $(T+1)/(2T)$. This is a work-count model, not a prediction of exact latency; launch costs and masking remain.

6 What explains the speedup?

6.1 The baseline spends most of its time in the score product

We measured the three V0 stages with CUDA events in the `V0 stage-timing executable`. On both dense shapes, its outputs were finite and bitwise equal to V0, and the instrumented median was within 0.02% of the original. This run used five warmups and 30 samples without flushing the caches. The cold-cache Nsight captures below were separate runs with replay cache flushing.

Dense shape	QK	Softmax	PV	Pipeline
$(2, 8, 2048, 64)$	29.711 ms	1.286 ms	6.621 ms	37.619 ms
$(1, 8, 2048, 128)$	29.594 ms	0.645 ms	6.633 ms	36.871 ms

Table 5: V0 stage medians from a separate event-timing run. The score stage accounts for approximately 79% and 80% of pipeline latency. The two shapes change both batch and head dimension.

The score product accounts for roughly four fifths of the breakdown in Table 5. Removing the standalone softmax alone would save little time. In V0, adjacent key threads can access different key rows at a stride of D elements for the same feature. In V2, the block stages contiguous chunks of K cooperatively and warps reuse them from shared memory. This changes the memory access pattern of the dominant V0 stage.

6.2 Eliminating the workspace and reducing operand requests

For dense attention, the two products do about $4BHN^2D$ FLOPs. With R queries sharing a full BF16 key/value tile, key and value requests sum to about $4BHN^2D/R$ bytes across blocks. Ignoring the linear query and output terms, the ratio is R FLOP per requested byte: about 1 in V1 and 8 in V2. This is an operand-request model, not a measurement of DRAM traffic. DRAM has a separate ceiling based on bytes that actually reach device memory.

At dense $(2, 8, 2048, 64)$, the grid measures 76.06 ms in V1 and 7.54 ms in V2a. The 10.1× ratio includes reuse, warp-local reductions and the load schedule. V1's per-query requests follow from its block mapping and the traffic model. The stage timings locate the dominant work in V0. At fixed tile sizes, doubling N quadruples dense work and approximately quadruples operand requests. Fusion removes the quadratic allocation, which reaches 2 GiB in V0 at $B = 4$, $H = 8$, $N = 4096$.

6.3 What the cold-cache profile shows

At causal $(1, 8, 2048, 128)$, V2a requests an estimated 1.12 GB before cache reuse. The cold capture records 1.09 GB of L2 requests, using 32 bytes per sector, and just 17.4 MB of DRAM reads plus writes. The L2 read hit rate reaches 98.8% within the launch. At 17% achieved occupancy, schedulers issue on 38% of active cycles. The two largest stall categories are MIO queue pressure and short-scoreboard waits, at 1.52 and 1.25 waiting warps per issue cycle, respectively.

7 Layout and buffering trade-offs

7.1 Packing value reads has the larger effect

The value-access controls keep the query mapping and key swizzle fixed. One packs the BF16 values read by each lane while keeping the row-major value tile. The other swizzles that tile and retains scalar reads. These results use the separate interleaved ablation run; its absolute times are not mixed with the main-grid run. Table 6 reports the reduction in median latency relative to V2a. V2b combines both changes and adds compiler memory clobbers around the asynchronous-copy instructions.

Workload	Packed reads only	Value swizzle only	V2b
$N = 512$, $D = 64$, dense	22%	1%	40%
$N = 2048$, $D = 64$, dense	24%	2%	41%
$N = 2048$, $D = 128$, causal	33%	3%	39%

Table 6: Latency reductions against V2a in the value-access experiment. The 512-token case uses $B = 4$. The 2048-token cases use $B = 2$ and $B = 1$, respectively. All use eight heads.

Variant	Shared-load wavefronts (M)	Load bank conflicts (M)	DRAM (MB)
V2a	208.2	69.2	17.5
Packed only	104.3	0	17.4
Swizzle only	208.4	69.2	17.5
V2b	95.7	0	17.5

Table 7: Matched cold-cache counters at causal $(1, 8, 2048, 128)$. All four variants double-buffer keys. Table 8 compares V2b's single- and double-buffer schedules. DRAM counts reads plus writes.

Packing alone halves shared-load wavefronts at $D = 128$ and cuts them by about 22% in the profiled $D = 64$ case. At $D = 128$, double-buffered V2a and V2b both reach about 17% occupancy. Their cold-profile durations are 4.88 and 2.99 ms, respectively. The similar DRAM totals and lower shared-load work support an on-chip explanation for this matched case. Changes of 1–3% should be interpreted in light of timing variability.

7.2 A second key buffer trades residency for overlap

With one key buffer, switching the key copies from synchronous to asynchronous changes V2a latency by about 1% or less across the three tested workloads. Value copies remain asynchronous in both controls. Adding prefetch and a second key buffer reduces V2a latency by about 2–3% for $D = 64$. At $D = 128$, the asynchronous single-buffer schedule reduces latency by about 14% relative to double buffering.

V2b's single-buffer schedule reduces latency by 18–19% across the four $D = 128$ cases. At $D = 64$, changing the buffer count moves medians by about 1% or less. The two widths gain different amounts of residency (Table 8).

7.3 Shared-memory residency

With s KiB of shared memory per block and 1 KiB reserved by CUDA, the shared-memory ceiling is $\lfloor 100/(s+1) \rfloor$ blocks per SM on this GPU [3]. Registers and warp slots can lower that ceiling. For $D = 128$, the single-buffer launch nearly doubles achieved occupancy, giving the schedulers more warps to cover stalls.

Kernel	D	Shared KiB/block	Blocks/SM ceiling	Achieved occupancy
V1	64 / 128	33.0 / 33.3	2	—
V2b, double	64	28	3	49–50%
V2b, single	64	20	4	65–66%
V2b, double	128	54	1	17%
V2b, single	128	38	2	33%

Table 8: Shared-memory residency model and achieved occupancy from the 2048-token V2b captures, across both masks.

8 Design conclusions

For forward BF16/FP32 attention on the RTX 3070 Ti Laptop GPU with $R = 8$ and $D \in \{64, 128\}$, the V2 results favour packed value reads. At $D = 128$, the single-key-buffer schedule gives lower latency and higher residency. At $D = 64$, the two V2b schedules are close in latency. V3 provides a further measured speedup, with a smaller advantage for the shortest grid.

8.1 Attribution boundaries

Table 6 isolates packed reads and value swizzling in separate controls. Its V2b column measures the combination of both changes and copy-wrapper memory clobbers. In Table 8, the $D = 128$ single-buffer gain is consistent with higher residency. Buffer footprint and prefetch schedule change together. The V1-to-V2 comparison likewise combines cross-query reuse, reduction mapping and staging. V3 changes MMA, tile geometry, mapping, layout and staging together, so its speedup cannot be attributed solely to Tensor Cores.

8.2 Limitations

The study covers synthetic forward self-attention and the repository's implementations on one GPU. The V2 tiled measurements fix $R = 8$; V3 uses 64 query rows per block. Backward passes and library baselines such as cuBLAS or PyTorch SDPA fall outside this comparison. The FP64 oracle covers selected lengths through $N = 256$, with V1 and V3 checked at $N = 256$ in the archived main suite. The larger performance shapes contribute output finiteness evidence. The API covers dense and causal masks, whose rows always have at least one valid key.

V1's performance explanation comes from device-side timing, its block mapping and the operand-request model, with V0 stage timing for baseline context. The matched cold-cache evidence covers shared loads, DRAM traffic and occupancy for V2a/V2b. The full V2a capture characterises L2 behaviour and scheduler stalls for that kernel. A separate V3 cold profile at dense $(2, 8, 2048, 64)$ records 4,194,304 tensor-pipe instructions, approximately 42.9% tensor-pipe active cycles, 23.3% achieved occupancy and approximately 16.75 MB of DRAM reads plus writes. These describe that one profiled launch and do not establish the isolated cause of the V3 speedup.

9 Reproducibility and contributions

Item	Recorded environment
Host	Intel Core i7-12700H, Ubuntu 22.04.5 LTS (capture metadata)
GPU and driver	RTX 3070 Ti Laptop GPU (sm_86), NVIDIA 580.65.06
Toolkit and profiler	NVCC 13.2.86, Nsight Compute 2026.1.1
Build	CMake 3.22 or newer required, Release, C++17/CUDA17, target sm_86.
Power and clocks	115 W GPU power cap in the device snapshot. Event harnesses request no clock lock. Nsight captures use default clock control.

9.1 Commands and evidence files

The ablation archive records base `5ebfe8a` and a worktree patch. The later main grid and V3 implementation come from a different working-tree state. The worktree was modified at report preparation. The local evidence manifest records hashes of the data used for this draft; it does not by itself establish the source revision of each measurement.

Run from the repository root. The main correctness target also runs the benchmark. The ablation executable runs its own FP64 checks before timing. Existing output files in the selected benchmark directory are overwritten.

Build and run

```
cmake -S . -B build -DCMAKE_BUILD_TYPE=Release -DCMAKE_CUDA_ARCHITECTURES=86
cmake --build build -j
ctest --test-dir build --output-on-failure
./build/bench_flash_attention results/raw/flash_attention
./build/profile_attention_v0
./build/ablation_experiments 30 1
python3 kernels/ablation/summarize.py
```

Evidence. The main grid is stored under `results/raw/flash_attention/` as raw timing, summary and accuracy CSV files. V0 stage medians are in `results/raw/v0_stage_timing.csv`. The B/C/D ablation CSVs and NCU captures are under `kernels/ablation/result/`. The separate V3 cold profile is under `results/ncu/`. Selected CSVs and the V2a and V3 cold reports are copied into the accompanying `evidence/` directory. Generated source paths are ignored by Git, so this evidence package must be distributed or archived alongside the final submission for independent inspection. Nsight replay and cache control are described in [5].

Project repository. Group 3 CUDA attention repository

Member responsibilities in the project proposal. Member A: CPU FP64 oracle, correctness suite and numerical analysis; cross-review of V2 tails and optional causal indexing. Member B: V0/V1, FLOP, workspace and traffic models, and fusion analysis; cross-review of benchmark metric definitions. Member C: V2 tiling, K/V reuse, vectorisation and shared-memory layout; cross-review of oracle and output indexing. Member D: asynchronous copy and staging, automation, profiling and plots; cross-review of synchronisation and resource analysis. Member C also implemented the V3 Tensor Core kernel. The A–D assignments follow the proposal, with V3 added to C's responsibilities.

AI declaration. OpenAI Codex assisted in drafting the report, checking descriptions against the repository, analysing the archived results and formatting the LaTeX. The group is responsible for verifying all numerical results, tables, figures and references against the source data and profiler captures.

References

[1] T. Dao et al. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. NeurIPS, 2022.
[2] T. Dao. FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning. ICLR, 2024.
[3] NVIDIA. Ampere GPU Architecture Tuning Guide.
[4] NVIDIA. CUDA C++ Best Practices Guide, shared-memory banks.
[5] NVIDIA. Nsight Compute Profiling Guide, replay and cache control.

A V3 grid parallelism

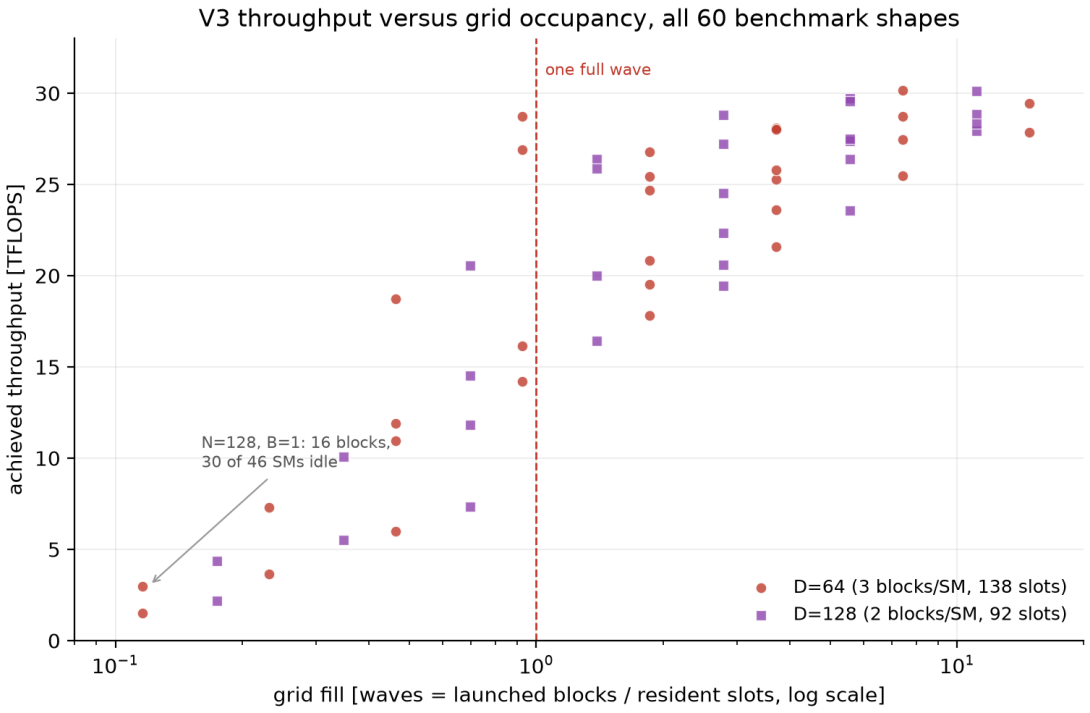


Figure 2: V3 throughput and estimated grid fill for the main grid. Estimated waves are launched blocks divided by the theoretical resident-block slots on 46 SMs, using three slots per SM for $D = 64$ and two for $D = 128$. Each point is a shape and mask. This is a resource-based estimate rather than a direct measurement of concurrent active SMs.

Larger grids tend to attain higher throughput, but the scatter also reflects sequence length, mask, head dimension and workload. One wave is a reference point, not a demonstrated saturation threshold.